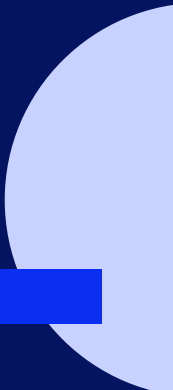
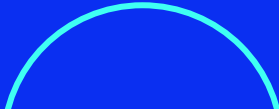


Introducción a MongoDB

Módulo 5



Cambiar la configuración del conjunto de réplicas

Cambiar la configuración del conjunto de réplicas

Con la función **rs.reconfig** ejecutada sobre el nodo primario, se puede modificar la configuración del conjunto réplica set.

Estableciendo un nuevo valor para la variable **config**, es posible fijar la prioridad que tiene cada nodo:

```
config = { _id:"m101", members: [  
  { _id:0, host:'localhost:27017', priority:3 },  
  { _id:1, host:'localhost:27018', priority:2 },  
  { _id:2, host:'localhost:27019', priority:1 }  
]};  
rs.reconfig(config)
```

Nota: cuanto mayor sea el valor de **priority**, mayor será la prioridad.



```
m101:PRIMARY> config = { _id:"m101", members: [ { _id:0, host:'localhost:27017', priority:3 }, { _id:1, host:'localhost:27018', p
riority:2 }, { _id:2, host:'localhost:27019', priority:1 } ]};
{
  "_id" : "m101",
  "members" : [
    {
      "_id" : 0,
      "host" : "localhost:27017",
      "priority" : 3
    },
    {
      "_id" : 1,
      "host" : "localhost:27018",
      "priority" : 2
    },
    {
      "_id" : 2,
      "host" : "localhost:27019",
      "priority" : 1
    }
  ]
}
```

```
m101:PRIMARY> rs.reconfig(config)
{
  "ok" : 1,
  "$clusterTime" : {
    "clusterTime" : Timestamp(1653138524, 1),
    "signature" : {
      "hash" : BinData(0,"AAAAAAAAAAAAAAAAAAAAAAAAAAAA="),
      "keyId" : NumberLong(0)
    }
  },
  "operationTime" : Timestamp(1653138524, 1)
}
```

Luego, con **rs.conf()** se puede verificar el cambio de configuración del conjunto de réplicas:

```
m101:PRIMARY> rs.conf()
{
  "_id" : "m101",
  "version" : 2,
  "term" : 1,
  "members" : [
    {
      "_id" : 0,
      "host" : "localhost:27017",
      "arbiterOnly" : false,
      "buildIndexes" : true,
      "hidden" : false,
      "priority" : 3,
      "tags" : {
      },
      "secondaryDelaySecs" : NumberLong(0),
      "votes" : 1
    },
  ],
}
```

```
{
  "_id" : 1,
  "host" : "localhost:27018",
  "arbiterOnly" : false,
  "buildIndexes" : true,
  "hidden" : false,
  "priority" : 2,
  "tags" : {

  },
  "secondaryDelaySecs" : NumberLong(0),
  "votes" : 1
},
```

```
{
  "_id" : 2,
  "host" : "localhost:27019",
  "arbiterOnly" : false,
  "buildIndexes" : true,
  "hidden" : false,
  "priority" : 1,
  "tags" : {
  },
  "secondaryDelaySecs" : NumberLong(0),
  "votes" : 1
},
```

...

```
"protocolVersion" : NumberLong(1),
"writeConcernMajorityJournalDefault" : true,
"settings" : {
  "chainingAllowed" : true,
  "heartbeatIntervalMillis" : 2000,
  "heartbeatTimeoutSecs" : 10,
  "electionTimeoutMillis" : 10000,
  "catchUpTimeoutMillis" : -1,
  "catchUpTakeoverDelayMillis" : 30000,
  "getLastErrorModes" : {

  },
  "getLastErrorDefaults" : {
    "w" : 1,
    "wtimeout" : 0
  },
  "replicaSetId" : ObjectId("6288de46898db8af955e6b95")
}
```


Ver información de replicación para nodos de respaldo

Con la función **db.printSecondaryReplicationInfo()** sobre el nodo secundario se pueden verificar los nodos secundarios, como se muestra a continuación:

```
m101:SECONDARY> db.printSecondaryReplicationInfo()

source: localhost:27018
  syncedTo: Sat May 21 2022 10:11:16 GMT-0300
  0 secs (0 hrs) behind the primary
source: localhost:27019
  syncedTo: Sat May 21 2022 10:11:16 GMT-0300
  0 secs (0 hrs) behind the primary
```



Prueba de conjunto de réplica

Copia de prueba

Se insertan 10000 datos de usuarios en el nodo Primario:

```
for(var i=1; i<=10000; i++) {  
    db.usuarios.insert({usuario_id:i, nombre:"nombre_"+i})  
}
```





```
Cmder

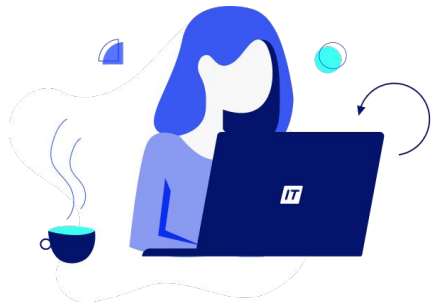
Cannot use non-local read concern until replica set is finished initializing.,"nextWakeupMillis":15000}}
{"t":{"$date":"2022-05-21T09:37:39.507-03:00"},"s":"I",  "c":"-",          "id":4939300, "ctx":"monitoring-keys-fo
r-HMAC","msg":"Failed to refresh key cache","attr":{"error":"NotYetInitialized: Cannot use non-local read concer
n until replica set is finished initializing.,"nextWakeupMillis":15200}}
{"t":{"$date":"2022-05-21T09:37:54.709-03:00"},"s":"I",  "c":"-",          "id":4939300, "ctx":"monitoring-keys-fo

m101:PRIMARY> for(var i=1; i<=10000; i++) {      db.usuarios.insert({usuario_id:i, nombre:"nombre_"+i}) }
WriteResult({ "nInserted" : 1 })
m101:PRIMARY> db.usuarios.count()
10000
m101:PRIMARY> db.usuarios.findOne()
{
  "_id" : ObjectId("6288e719ba04b4e486c99407"),
  "usuario_id" : 1,
  "nombre" : "nombre_1"
}
m101:PRIMARY> |
```



mongod.exe mongo.exe mongod.exe [*] mongo.exe mongod.exe [*] mongo.exe

Comprobar que los datos del cliente se han sincronizado en los nodos secundarios: se debe utilizar la función **rs.secondaryOk()**, en cada nodo secundario, para habilitar dicha operación, como se muestra a continuación:



```
m101:SECONDARY> rs.secondaryOk()
m101:SECONDARY> db.usuarios.count()
10000
m101:SECONDARY> db.usuarios.findOne()
{
  "_id" :
  ObjectId("6288e719ba04b4e486c99407"),
  "usuario_id" : 1,
  "nombre" : "nombre_1"
}
m101:SECONDARY>
```

En ambos secundarios, se puede confirmar que están sincronizados con los datos del nodo primario.

Prueba de conmutación por error

Se detendrá la instancia primaria (con **Control-C** sobre la consola) y se saldrá del *shell*:

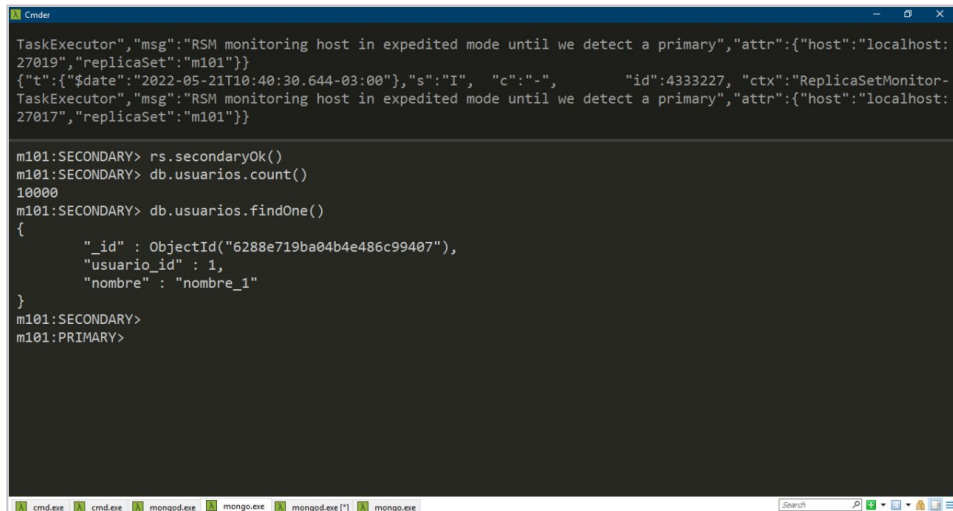
```
Cmder
{"t":{"$date":"2022-05-21T10:36:00.679-03:00"},"s":"I", "c":"-", "id":4784931, "ctx":"consoleTerminate",
"msg":"Dropping the scope cache for shutdown"}
{"t":{"$date":"2022-05-21T10:36:00.680-03:00"},"s":"I", "c":"FTDC", "id":4784926, "ctx":"consoleTerminate",
"msg":"Shutting down full-time data capture"}
{"t":{"$date":"2022-05-21T10:36:00.680-03:00"},"s":"I", "c":"FTDC", "id":20626, "ctx":"consoleTerminate",
"msg":"Shutting down full-time diagnostic data capture"}
{"t":{"$date":"2022-05-21T10:36:00.683-03:00"},"s":"I", "c":"CONTROL", "id":20565, "ctx":"consoleTerminate",
"msg":"Now exiting"}
{"t":{"$date":"2022-05-21T10:36:00.683-03:00"},"s":"I", "c":"CONTROL", "id":23138, "ctx":"consoleTerminate",
"msg":"Shutting down", "attr":{"exitCode":12}}

C:\
λ |

m101:PRIMARY> exit
bye
```



Luego, se observa en los secundarios, que uno de ellos se convierte en el nuevo nodo primario:

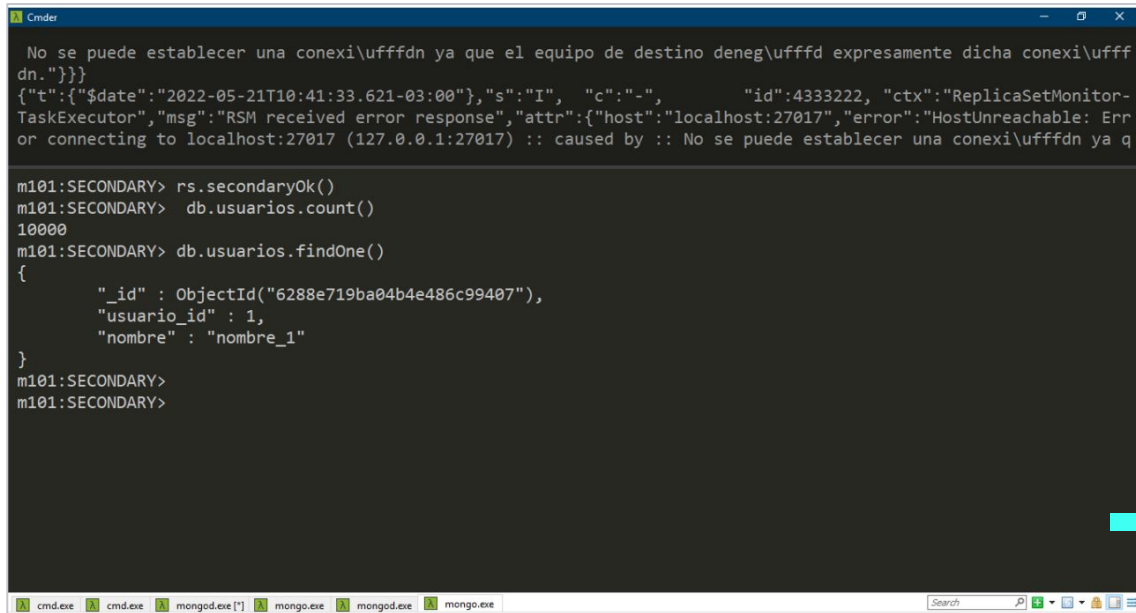


```
TaskExecutor","msg":"RSM monitoring host in expedited mode until we detect a primary","attr":{"host":"localhost:27019","replicaSet":"m101"}}
{"t":{"$date":"2022-05-21T10:40:30.644-03:00"},"s":"I",  "c":"-",          "id":4333227, "ctx":"ReplicaSetMonitor-TaskExecutor","msg":"RSM monitoring host in expedited mode until we detect a primary","attr":{"host":"localhost:27017","replicaSet":"m101"}}

m101:SECONDARY> rs.secondaryOk()
m101:SECONDARY> db.usuarios.count()
10000
m101:SECONDARY> db.usuarios.findOne()
{
  "_id" : ObjectId("6288e719ba04b4e486c99407"),
  "usuario_id" : 1,
  "nombre" : "nombre_1"
}
m101:SECONDARY>
m101:PRIMARY>
```

En este caso, es el nodo que está en el puerto 27018 que es el que tiene más prioridad.

El 27019 permanecerá como secundario:



```
Cmder

No se puede establecer una conexi\ufffdn ya que el equipo de destino deneg\ufffd expresamente dicha conexi\ufffd
dn."}}}}
{"t":{"$date":"2022-05-21T10:41:33.621-03:00"},"s":"I", "c":"-", "id":4333222, "ctx":"ReplicaSetMonitor-
TaskExecutor","msg":"RSM received error response","attr":{"host":"localhost:27017","error":"HostUnreachable: Err
or connecting to localhost:27017 (127.0.0.1:27017) :: caused by :: No se puede establecer una conexi\ufffdn ya q

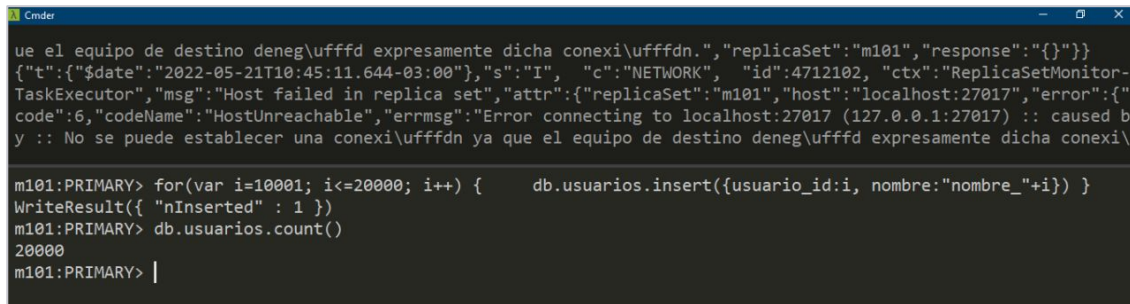
m101:SECONDARY> rs.secondaryOk()
m101:SECONDARY> db.usuarios.count()
10000
m101:SECONDARY> db.usuarios.findOne()
{
  "_id" : ObjectId("6288e719ba04b4e486c99407"),
  "usuario_id" : 1,
  "nombre" : "nombre_1"
}
m101:SECONDARY>
m101:SECONDARY>
```



Sobre el nuevo nodo primario (27018), se insertan 10000 documentos más:

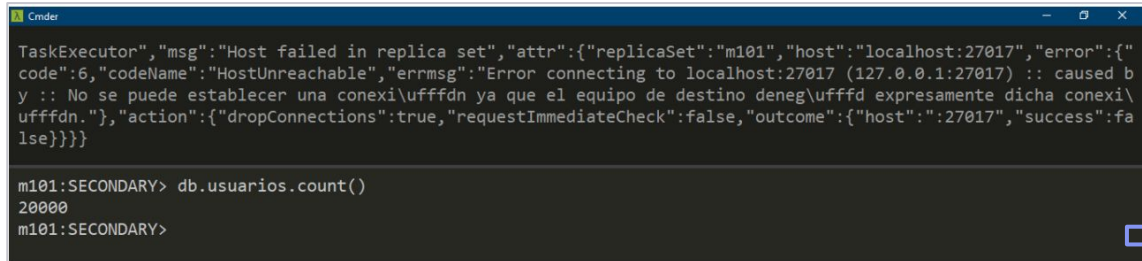
```
for(var i=10001; i<=20000; i++) {  
    db.usuarios.insert({usuario_id:i, nombre:"nombre_"+i})  
}
```

Se debe verificar, en el nuevo primario, que los documentos estén insertados:



```
m101:PRIMARY> for(var i=10001; i<=20000; i++) {    db.usuarios.insert({usuario_id:i, nombre:"nombre_"+i}) }  
WriteResult({ "nInserted" : 1 })  
m101:PRIMARY> db.usuarios.count()  
20000  
m101:PRIMARY> |
```


Se hace lo mismo en el secundario:



```
TaskExecutor", "msg": "Host failed in replica set", "attr": {"replicaSet": "m101", "host": "localhost:27017", "error": {"code": 6, "codeName": "HostUnreachable", "errmsg": "Error connecting to localhost:27017 (127.0.0.1:27017) :: caused by :: No se puede establecer una conexi\u00f3n ya que el equipo de destino deneg\u00f3 expresamente dicha conexi\u00f3n."}, "action": {"dropConnections": true, "requestImmediateCheck": false, "outcome": {"host": "27017", "success": false}}}}

m101:SECONDARY> db.usuarios.count()
20000
m101:SECONDARY>
```

Los datos se han replicado en el nodo secundario:

Se debe volver a iniciar el nodo **localhost:27017**, y verificar que se inicia como secundario. Como tiene la mayor prioridad de elección, se elige automáticamente como Primario. En unos instantes quedará con su información sincronizada, como se muestra en la siguiente pantalla.

```
connection accepted", "attr": {"remote": "127.0.0.1:1809", "uuid": "77ae1124-ac69-4e6a-86ad-45146b8ad5cf", "connectionId": 54, "connectionCount": 14}}
{"t": {"$date": "2022-05-21T10:35:49.653-03:00"}, "s": "I", "c": "NETWORK", "id": 51800, "ctx": "conn54", "msg": "client metadata", "attr": {"remote": "127.0.0.1:1809", "client": "conn54", "doc": {"driver": {"name": "NetworkInterfaceTL", "

m101:SECONDARY>
m101:PRIMARY> db.usuarios.count()
20000
m101:PRIMARY>
```

El servidor primario anterior (27018) volvió a ser secundario:

```
code":6,"codeName":"HostUnreachable","errmsg":"Error connecting to localhost:27017 (127.0.0.1:27017) :: caused by :: No se puede establecer una conexi\u00f3n ya que el equipo de destino deneg\u00f3 expresamente dicha conexi\u00f3n.", "action":{"dropConnections":true,"requestImmediateCheck":true}}
{"t":{"$date":"2022-05-21T10:50:36.163-03:00"},"s":"I", "c":"-", "id":4333227, "ctx":"ReplicaSetMonitor-TaskExecutor","msg":"RSM monitoring host in expedited mode until we detect a primary","attr":{"host":"localhost:

m101:PRIMARY> for(var i=10001; i<=20000; i++) { db.usuarios.insert({usuario_id:i, nombre:"nombre_"+i}) }
WriteResult({ "nInserted" : 1 })
m101:PRIMARY> db.usuarios.count()
20000
m101:PRIMARY>
m101:SECONDARY>
```

**¡Sigamos
trabajando!**